Tarea 2: Torneo de Komikan Concurrente

Sistemas Operativos

David Benjamin Fuentes Castro

3 de noviembre de 2026

1. Diseño del Esquema de Sincronización

Para abordar la naturaleza altamente concurrente del torneo de Komikan en tiempo real, se diseñó una arquitectura de sincronización de dos niveles: a nivel de servidor (Matchmaker) y a nivel de tablero.

- **Nivel Servidor:** Se utilizó un semáforo POSIX (`sem_t`) inicializado con el valor máximo de mesas concurrentes (M). Esto garantiza que el emparejamiento de jugadores bloquee su ejecución pasivamente si la capacidad máxima es alcanzada, despertando automáticamente cuando un hilo Juez finaliza su partida y ejecuta `sem_post`.
- **Nivel Tablero (Fine-grained locking):** Se optó por un modelo de bloqueo fino. En lugar de usar un único candado global (que serializaría la ejecución y anularía la ventaja de usar hilos), cada una de las 29 casillas posee su propio `pthread_mutex_t`. Adicionalmente, el estado global de la partida (contadores de victoria y bandera de término) está protegido por un candado dedicado (`mutex_estado`).

2. Análisis de Riesgos de Concurrencia

Dado que 13 hilos (1 Puma, 12 Perros) compiten simultáneamente en un grafo interconectado, se identificaron y resolvieron los siguientes riesgos de concurrencia:

- **Deadlocks (Abrazos Mortales):** Si dos piezas intentan moverse hacia casillas cruzadas al mismo tiempo bloqueándolas con `pthread_mutex_lock`, ocurriría un interbloqueo. Se previno este escenario obligando a los hilos a usar `pthread_mutex_trylock` al evaluar las casillas de destino (y de salto). Si el candado está ocupado, el hilo no se bloquea, sino que desiste de esa rama de ejecución, garantizando que el sistema nunca se congele.
- **Livelocks y Starvation (Inanición):** Al basar la lógica en `trylock`, existe el riesgo teórico de que los hilos entren en un ciclo infinito de colisiones simultáneas fallidas (livelock). Para evitarlo y asegurar equidad, se incorporaron retardos aleatorios en cada ciclo de vida del hilo utilizando `nanosleep` (50 a 300 ms). Esto introduce desincronización temporal (jitter), asegurando que las piezas no colisionen de forma determinista y perpetua, dando a todas las piezas la oportunidad de acceder a los recursos (evitando la inanición).

3. Evaluación y Justificación

El diseño implementado privilegia el **máximo nivel de concurrencia posible** por sobre la simplicidad del código.

- **Rendimiento (Throughput):** El candado por casilla permite que múltiples perros se muevan en distintas zonas del tablero de manera estrictamente paralela, aprovechando el paralelismo de la CPU.
- **Complejidad Controlada:** El mayor desafío técnico de esta aproximación es la lógica de captura del Puma, que requiere adquirir de forma atómica tres candados simultáneos (origen, perro a capturar y destino). El enfoque de `trylock` anidado logró resolver esta transacción crítica sin sacrificar el throughput del resto del tablero y previniendo deadlocks estructurales.

En conclusión, la infraestructura construida proporciona un servidor resiliente y un motor de juego robusto, cumpliendo a cabalidad con la especificación asimétrica y de tiempo real exigida.